

PROJETO A3 CLEAN CODE

Profº Robson Calvetti

Andressa Pereira - 823213904; Júlia Rocha - 823214680; Lucas Marzocca - 823116813; Matheus Guimarães - 823141914;
Marcos Santos - 82327399; Mylena Rocha - 824144075; Samuel da Costa - 824147380



CÓDIGO LEGADO

SISTEMA DE GERENCIAMENTO DE BIBLIOTECA

FUNCIONALIDADE

O sistema permitia cadastrar livros, usuários e controlar empréstimos.

DEFICIÊNCIAS

- Uso intenso de variáveis globais, gerando estado **descontrolado**
- Funções acumulando múltiplas responsabilidades (validação, criação, persistência, mensagens)
- Código **duplicado** em buscas, validações e verificações
- Nomes pouco descritivos e uso de **magic numbers**
- Falta de tratamento de exceções e ausência de orientação a objetos

```
def adicionarLivro(titulo, autor, isbn, ano):  
    global contador_livros  
  
    if titulo == "" or autor == "" or isbn == "":  
        print("Erro: campos obrigatórios")  
        return False  
  
    for livro in livros:  
        if livro['isbn'] == isbn:  
            print("Livro já existe")  
            return False  
  
    if len(isbn) != 13 and len(isbn) != 10:  
        print("ISBN inválido")  
        return False
```

CÓDIGO REFATORADO: MELHORIAS APLICADAS

+

01

Realização de testes unitários prévios

14 testes unitários automatizados via GitHub Actions (Python 3.11 + unittest) validam operações CRUD, empréstimos, devoluções, validações de dados, duplicidade, persistência JSON e exceções, assegurando integridade funcional pós-refatoração POO.

02

Modularização e Separação de Responsabilidades

O código foi dividido em múltiplos arquivos, cada um com uma responsabilidade clara. Isso melhora a legibilidade, manutenibilidade e testabilidade, seguindo princípios como SRP e Clean Architecture.

03

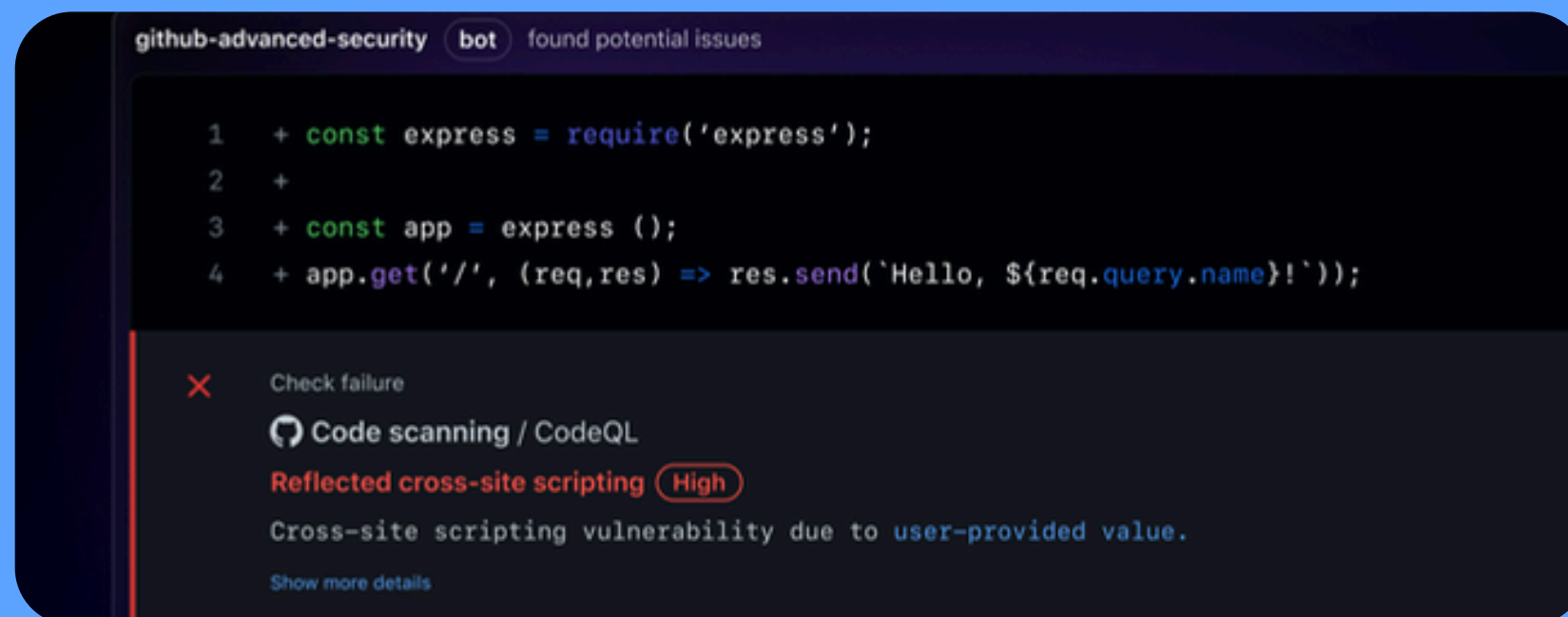
Refatoração de Estruturas e Persistência

Variáveis globais foram removidas e transformadas em atributos de instância dentro das classes. Além disso, a persistência foi refatorada utilizando serialização via vars(), garantindo um sistema mais robusto e fácil de estender.

04

Design Pattern e Testes

Implementação do Factory Method para criação de objetos, centralizando a lógica de construção e reduzindo acoplamento. Também foram adicionados testes unitários, assegurando a confiabilidade do sistema e a facilidade de futuras modificações sem comprometer a funcionalidade.



The screenshot shows a GitHub Actions workflow failure. At the top, a message from 'github-advanced-security bot' states 'found potential issues'. Below this, a code snippet is displayed:

```
1 + const express = require('express');
2 +
3 + const app = express ();
4 + app.get('/', (req,res) => res.send(`Hello, ${req.query.name}!`));
```

Below the code, a 'Check failure' section is shown with the title 'Code scanning / CodeQL'. The failure is labeled 'Reflected cross-site scripting' with a 'High' severity. The description reads: 'Cross-site scripting vulnerability due to user-provided value.' A link to 'Show more details' is provided at the bottom.

NOSSA EXPERIÊNCIA E APRENDIZADOS

Adotamos uma organização baseada em **Scrum**, com ciclos curtos e alinhamentos frequentes para dividir as tarefas e acompanhar o progresso. Isso ajudou a entender o **código legado**, resolver conflitos no Git e escolher o **Design Pattern adequado**. A experiência mostrou como **comunicação** constante e pequenas iterações tornam o trabalho em grupo mais **eficiente**.



01

Interações curtas tornam o avanço mais visível

Ao dividir o trabalho em etapas menores, conseguimos acompanhar melhor a evolução do projeto e ajustar rapidamente o que fosse necessário, mantendo todos alinhados.

02

Código limpo depende de controle e versão

O uso do Git trouxe confiabilidade ao processo, permitindo registrar cada alteração com segurança, recuperar versões anteriores sem risco de perda e revisar o código de forma clara entre os membros. Essa rastreabilidade deixou o fluxo mais organizado e reduziu erros durante o desenvolvimento.

A IMPORTÂNCIA DO CLEAN CODE

POR QUE O CLEAN CODE IMPORTA:

- Facilita leitura e entendimento do código
- Reduz tempo de manutenção e correção de bugs
- Ajuda novos desenvolvedores a compreender o sistema rapidamente
- Diminui a dívida técnica e aumenta a vida útil do projeto

IMPACTOS REAIS OBSERVADOS:

- Funções ficaram significativamente menos complexas
- Novas funcionalidades passaram a ser implementadas em minutos
- Testes unitários agora protegem o sistema contra regressões

OBSERVAÇÃO DO GRUPO:

"Aprendemos que Clean Code não é só técnica, é postura profissional. É o que diferencia manutenção caótica de evolução consistente."

CITAÇÃO (ROBERT C. MARTIN):

"Qualquer tolo pode escrever código que um computador entenda. Bons programadores escrevem código que humanos entendam."